

Labo 11 – Verslag CSS Grid

opleidingsonderdeel **Web Development I**

Bachelor in de **toegepaste informatica**

campus **Kortrijk**

Tibo Dewanckele

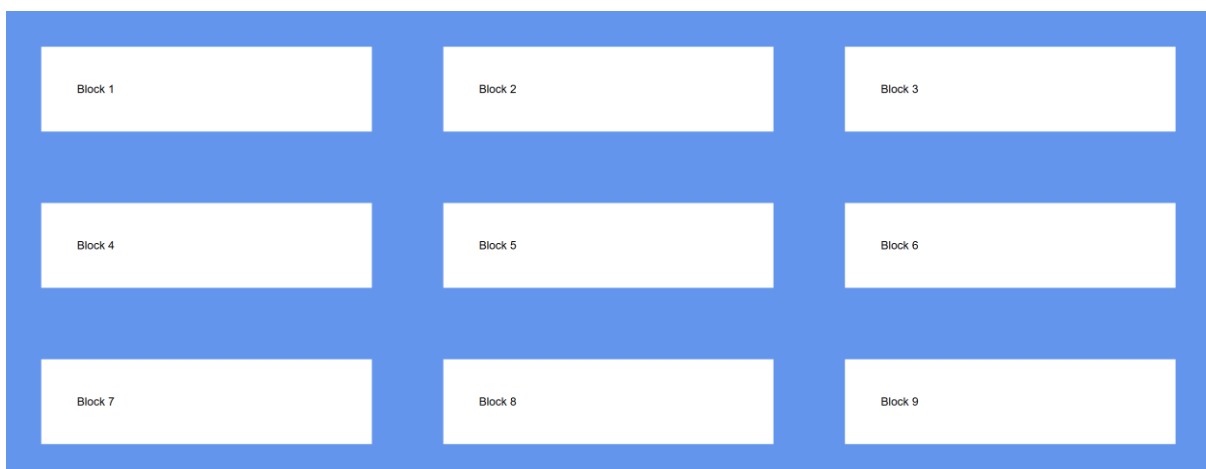
docent: **Pim Debaere**

academiejaar **2025–2026**

Inhoud

Opdracht 1.....	2
Opdracht 2.....	3
Opdracht 3.....	3
Opdracht 4.....	4
Opdracht 5.....	6
Opdracht 6.....	7
Aparte oefening.....	10

Opdracht 1



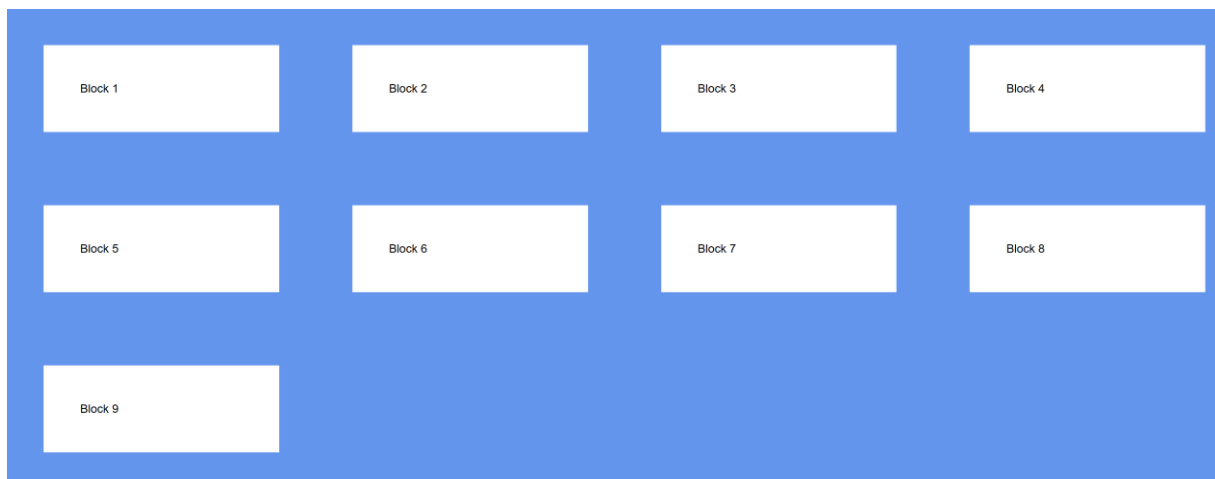
```
.container{  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr;  
  background-color: cornflowerblue;  
}
```

Via grid-template-columns geef je mee hoeveel kolommen door het aantal keer dat je een grootte opgeeft, hier heb ik drie keer een kolom van 1 fractionele unit.

De display moet ook op grid staan om dit te doen werken.

Er worden 3 rijen gegenereerd omdat er 9 blokken zijn verdeeld over 3 kolommen op elke rij.

Opdracht 2

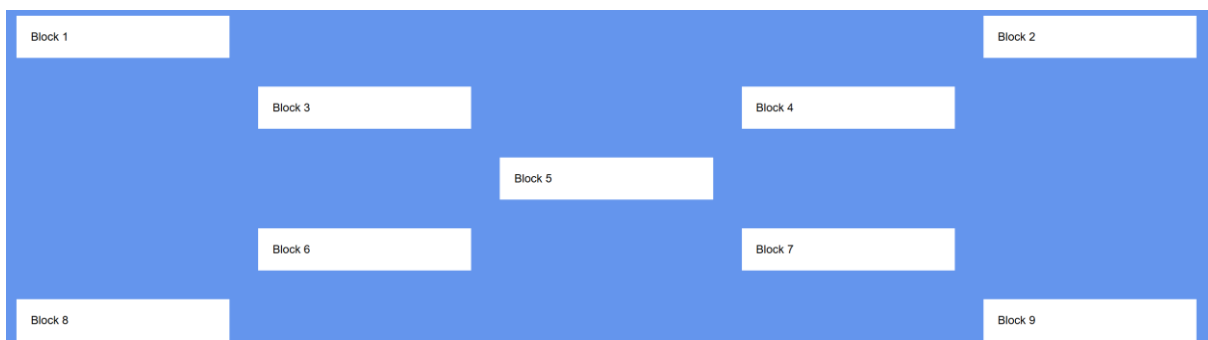


```
.container{  
  background-color: cornflowerblue;  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr 1fr;  
  grid-template-rows: 1fr 1fr 1fr;  
}
```

Dit is een grid van 4x3, er worden nog steeds 9 blokken getoond. Er wordt gewoon eerder gestopt met het vullen van de vakken van het grid.

Ook het aantal rijen konden we hier nu definiëren door grid-template-rows en opnieuw de hoogte van elke rij ook.

Opdracht 3



```
.container{  
  display:grid;  
  background-color: cornflowerblue;  
  grid-template-columns: 1fr 1fr 1fr 1fr 1fr;  
  grid-template-rows: 1fr 1fr 1fr 1fr 1fr;  
}
```

We geven mee dat we een grid willen van 5x5

```

.block.block2{
  grid-column-start: 5;
}
.block.block3{
  grid-column-start: 2;
}
.block.block4{
  grid-column-start: 4;
}
.block.block5{
  grid-column-start: 3;
}
.block.block6{
  grid-column-start: 2;
}
.block.block7{
  grid-column-start: 4;
}
.block.block8{
  grid-column-start: 1;
}
.block.block9{
  grid-column-start: 5;
}

```

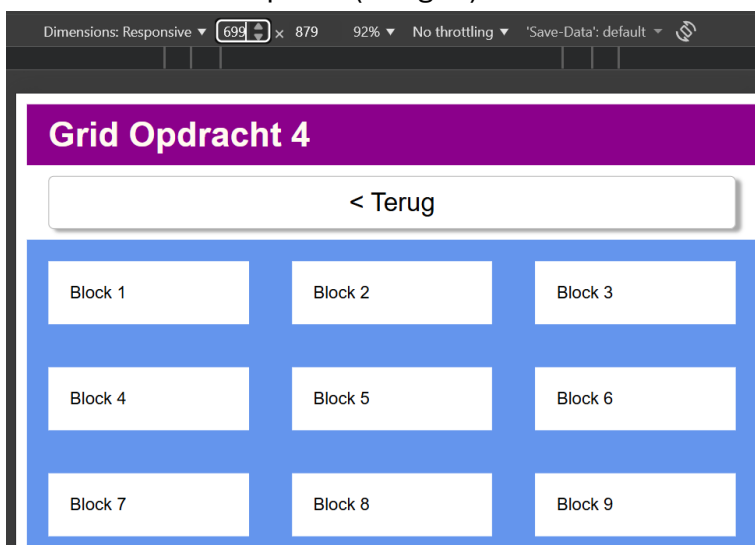
block 1 moet niet aangegeven worden waar hij moet staan omdat hij standaard als eerste zal worden geplaatst in het grid, omdat hij als eerste in de HTML staat.

De andere rijen geven we iedere keer mee waar ze in de kolom moeten starten, omdat we ofwel verder in dezelfde rij gaan of naar een lagere kolom in de volgende rij moeten gaan, moeten we niet opgeven in welke rij ze moeten staan.

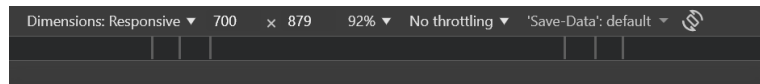
Via grid-column-start geef je mee in welke kolom de block moet starten, omdat we niets anders meegeven gaat hij alleen die kolom vullen.

Opdracht 4

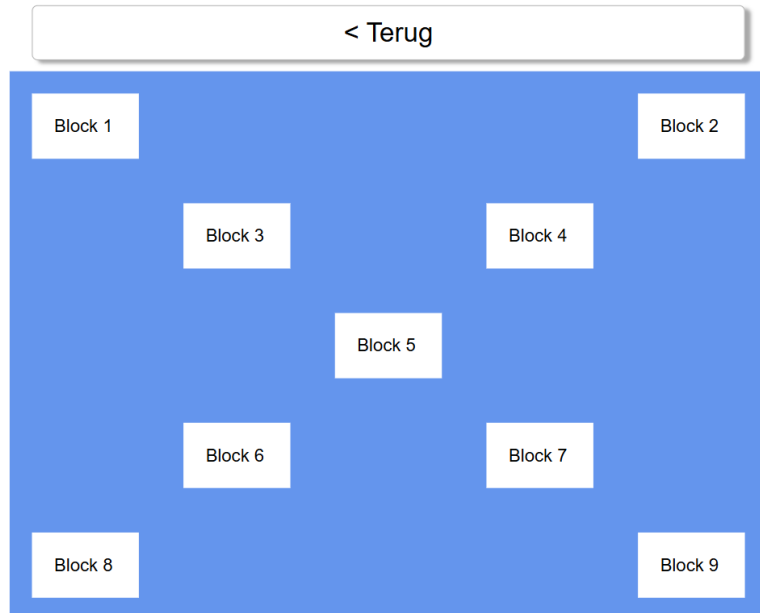
Breedte onder 700 pixels (3x3 grid)



Breedte vanaf 700 pixels (5x5 grid) (css overgenomen van opdracht 3)



Grid Opdracht 4



```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

In de HTML is dit nog steeds nodig om te zorgen dat de media query kan werken.

```
.container{
  display:grid;
  background-color: cornflowerblue;
  grid-template-columns: 1fr 1fr 1fr;
  grid-template-rows: 1fr 1fr 1fr;
}
```

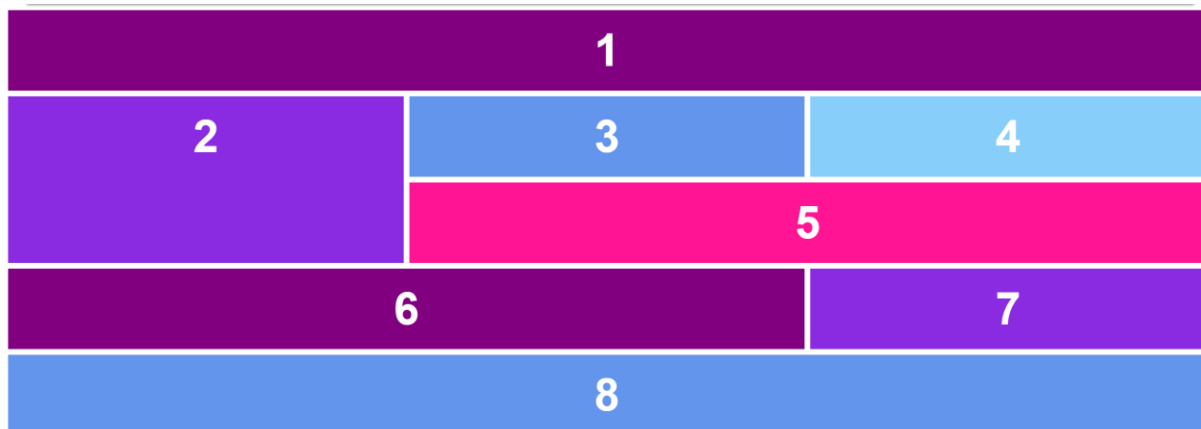
Omdat we mobile-first werken moeten we voor het kleinste starten en maken we dus een 3x3 grid.

```
@media only screen and (min-width: 700px){
  .container{
    grid-template-columns: 1fr 1fr 1fr 1fr 1fr;
    grid-template-rows: 1fr 1fr 1fr 1fr 1fr;
  }
}
```

Via een media query kunnen we dan zeggen vanaf een breedte van 700px wordt de grid 5x5.

Binnenin deze media query bevindt zich ook de css voor de blocks waar ze moeten starten.

Opdracht 5



```
.container{  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr;  
  grid-template-rows: 1fr 1fr 1fr;  
  gap: 0.5rem;  
}
```

We genereren een 3x3 grid omdat we kunnen zien dat er 3 verschillende elementen in een rij moesten zitten bij het voorbeeld.

Via gap kunnen we ruimte tussen de blocks creëren zonder margin te gebruiken.

```
.block.block1{  
  grid-column: span 3;  
  background-color: purple;  
}
```

Om te zorgen dat de volledige eerste rij wordt ingenomen door block1 geven we een span van 3 mee aan grid-column om te zorgen dat het 3 kolommen inneemt.

Standaard start hij vanaf het begin omdat hij als eerste in de HTML komt en als niets anders wordt meegegeven, wordt de volgorde daarin gevolgd.

```
.block.block2{  
  background-color: blueviolet;  
  grid-row: span 2;  
}
```

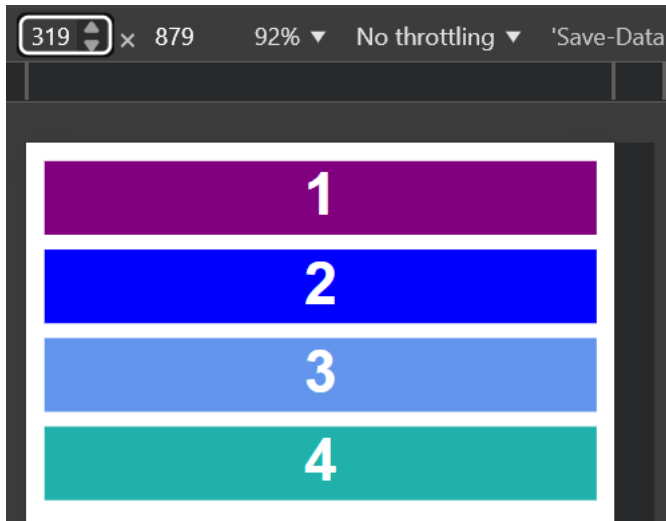
Om te zorgen dat block 2 hoger is en zich over 2 rijen bevindt kunnen we nu een span van 2 meegeven aan grid-row.

Hetzelfde wordt gedaan voor block5, block6 en block8 elk met hun eigen grootte.

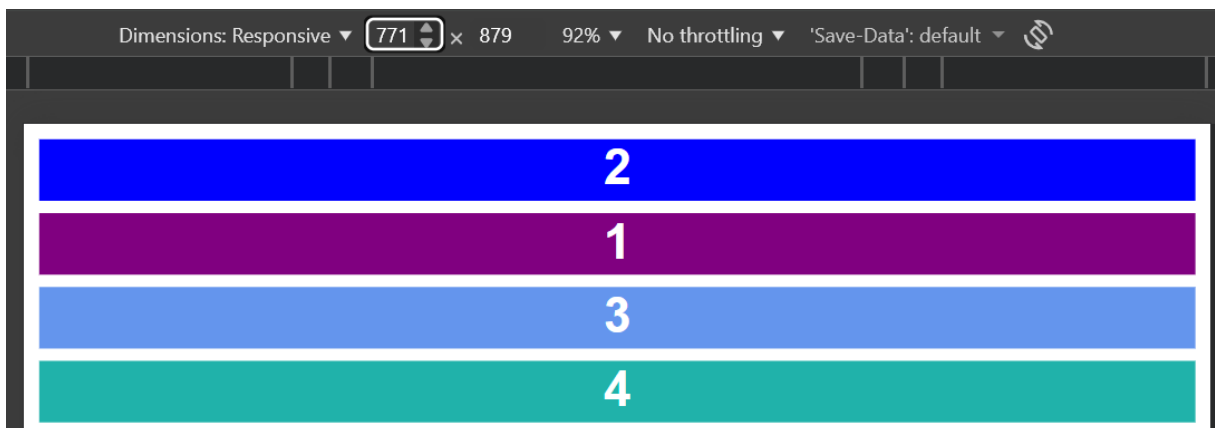
Als je dat dan doet, moet er ook geen startkolom of rij meegegeven worden om te zorgen dat het op de goede plaats komt omdat de volgorde nog steeds goed staat en degene dat we gewoon een achtergrondkleur hebben gegeven komen automatisch in een kolom terecht.

Opdracht 6

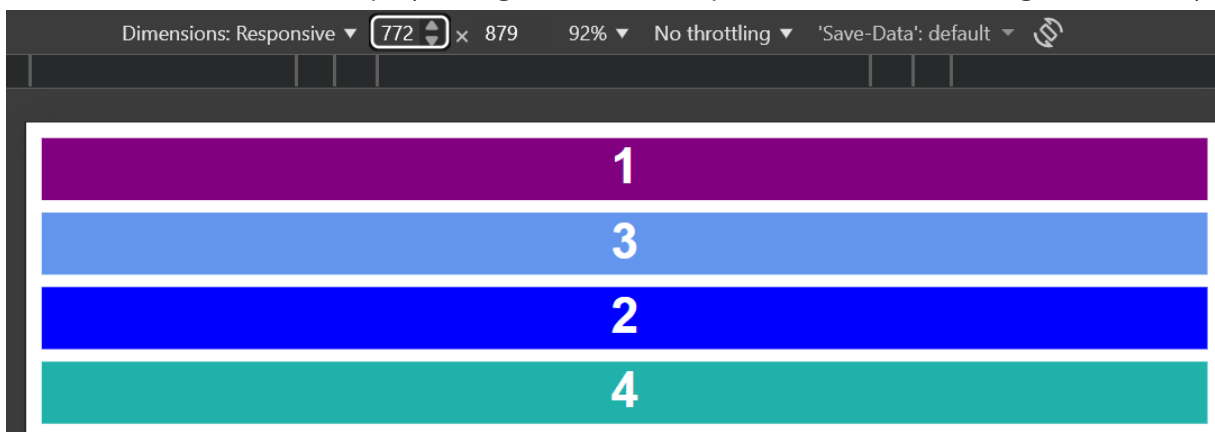
Als het minder breed is dan 320 pixels dan geef ik gewoon achtergrondkleur mee en pas ik geen volgorde of plaatsing aan.



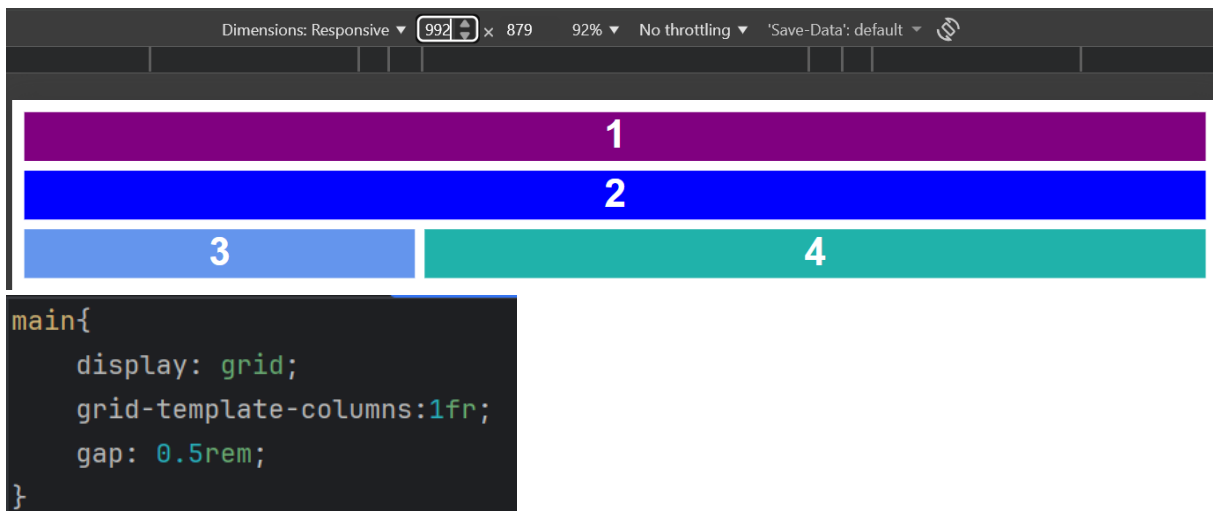
Vanaf een breedte van 320px (2 en 1 wisselen van plaats)



Vanaf een breedte van 772px (1 terug bovenaan, 3 op eronder en 2 daar nog eens onder)



Vanaf een breedte van 992px (terug normale volgorde, 3 en 4 wel in dezelfde rij en 4 dubbel zo groot als 3)



Doordat we mobile-first werken starten we met de grid 1 kolom te geven want in het begin werken we met 1 kolom. Er is ook opnieuw een gap (ruimte tussen blocks) gedefinieerd.

```
.block1{
  background-color: purple;
}

.block2{
  background-color: blue;
}

.block3{
  background-color: cornflowerblue;
}

.block4{
  background-color: lightseagreen;
}
```

Op het kleinste scherm heb ik dus gewoon achtergrondkleuren meegegeven om te zorgen dat we de blocks duidelijk kunnen zien ook al zou het scherm kleiner zijn dan 320 pixels.


```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

Bovenstaande is nodig voor de media queries (staat in HTML)

```
@media only screen and (min-width: 320px){  
  .block1{  
    grid-row: 2;  
  }  
  
  .block2{  
    grid-row: 1;  
  }  
}
```

Vanaf een breedte van 320px wil ik 1 en 2 wisselen dus zet ik de grid-row van 1 op 2 en omgekeerd bij block2.

```
@media only screen and (min-width: 772px){  
  .block1{  
    grid-row: 1;  
  }  
  
  .block2{  
    grid-row: 3;  
  }  
  
  .block3{  
    grid-row: 2;  
  }  
}
```

Vanaf een breedte van 772px zetten we block1 terug op grid-row 1 omdat hij anders de regel van bij 320px zal gebruiken en dus op rij 2 zou blijven.

Block2 wordt dan op rij 3 gezet en block3 op rij 2.

```
@media only screen and (min-width: 992px){  
  main{  
    grid-template-columns: 1fr 2fr;  
  }  
}
```

Vanaf een breedte van 992px willen we 3 en 4 in dezelfde rij zetten dus moeten we het aantal kolommen aanpassen. Omdat 4 dubbel zo groot moet zijn als 3 maken we die kolom al 2fr ipv 1fr, zo zal 4 automatisch groter zijn omdat we 4 in de tweede kolom in die rij gaan zetten

```
.block1{
  grid-column: span 2;
}

.block2{
  grid-row: 2;
  grid-column: span 2;
}
```

We moeten bij zowel block1 als block2 dan wel meegeven dat ze zich over 2 kolommen moeten bevinden en dus opnieuw heel de rij innemen.

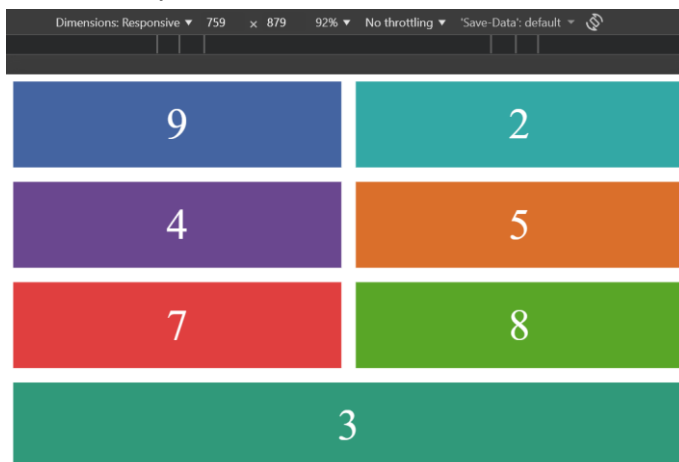
Block2 wordt dan ook weer terug in rij 2 gezet.

```
.block3{
  grid-row: 3;
}
```

Block3 wordt dan ook nog in rij 3 gezet en komt dan weer door de volgorde van de blokken voor block4 in de rij en daardoor komt deze terecht in de kolom van 1fr.

Aparte oefening

Onder 760 pixels



Vanaf 760 pixels



```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

We gaan dit opnieuw nodig hebben omdat we opnieuw met media queries werken.

```
body > div{
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 1rem;
}
```

Op het kleinste scherm willen we dus maximaal 2 blokken in een kolom waardoor dat we 2 kolommen genereren voor het grid. We steken er ook een gap bij om ruimte tussen de blokken te veroorzaken.

```
.item-1 {
  background: #b03532;
  display: none;
}
.item-6 {
  background: #3d8bb1;
  display: none;
}
```

Omdat item-1 en item-6 niet te zien moeten zijn op het kleinste scherm, zetten we display op none zodat ze niet te zien zijn. Dan kunnen we via media queries ze beide met dezelfde regel terug beschikbaar maken.

```
.item-3 {
  background: #30997a;
  grid-row: 4;
  grid-column: span 2;
}
```

item-3 bevindt zich op het kleinste scherm onderaan op rij 4 waardoor dat we grid-row op 4 zetten. Het moet zich ook over 2 kolommen plaatsen waardoor dat we een span 2 meegeven aan grid-column.

```
.item-9 {
  background: #4464a1;
  grid-row: 1;
}
```

We willen item-9 bovenaan zetten als eerste item, dus zetten we grid-row op 1 en dan gaat hij vooraan komen te staan. Alle andere items gaan 1 kolom opschuiven, als ze op de laatste kolom van een rij staan, gaan ze naar de eerste kolom van de volgende rij.

```
@media only screen and (min-width: 760px){
  body>div{
    grid-template-columns: 1fr 2fr 1fr;
    grid-template-rows: 1fr 1fr 1fr;
  }
}
```

Vanaf een breedte van 760px willen we een 3x3 grid waarbij de middelste kolom dubbel zo breed is als de andere kolommen, hierdoor geven we een grootte van 2fr mee aan de middelste kolom.

```
.item-1, .item-6 {  
  display: block;  
}
```

item-1 en 6 mogen terug zichtbaar worden, dus zetten we deze hier terug op display: block.

```
.item-3, .item-9{  
  grid-row: auto;  
  grid-column: auto;  
}
```

item-3 en 9 mogen terug op hun natuurlijke plaats staan, de volgorde van de HTML volgen, dus zetten we grid-row en -column op auto. Zo wordt de volgorde van HTML gevolgd.